# OzAgriMate Irrigation Architecture and API

## Quick Note

These sections document the architecture decisions, component and runtime views, and irrigation API contributed by Thomas Wight. The two irrigation endpoints pass 21 automated tests and a 13-request Postman run with 40 of 40 assertions passed. Architecture decisions remain Proposed pending team review.

## Design basis and scope

The submitted Assignment 2 report lists Reliability / Safety and Availability among its seven driving characteristics. ASR5 requires suspicious readings not to trigger automated actions, ASR12 requires controllers to keep the last approved schedule for 48 hours offline, ASR19 requires access revocation within one minute and constraint C3 requires 72-hour offline edge operation. ADR-01 places irrigation in the Irrigation and Alerts service, which owns requests, approved schedules and alert rules in its own Operations Store partition.

This contribution implements only irrigation request creation and pending-request replacement. Owner approval, dispatch to the Farm Edge Gateway and physical execution are planned capabilities; the prototype never implies that water has been applied.

### Contribution coverage

| Artefact | Scope and status |
|---|---|
| ADR-03 | Reliability / Safety with offline Availability; Proposed |
| ADR-07 | Event-fed farm access projection (data/dependency decision); Proposed |
| Farm Management component view | Matches Pulkit's implemented modules; outbox publisher marked proposed |
| Runtime view 2 | Irrigation request and processing; approval, dispatch and execution marked planned |
| Irrigation API | POST and PUT irrigation requests; local implementation on port 8003 |
| Testing evidence | 21 automated tests passed; Postman run of 13 requests, 40 assertions passed, zero failures and errors |

## ADR 03 Irrigation safety and offline continuity

| Field | Architectural decision record |
|---|---|
| Title | ADR-03 Enforce approval, safe limits and bounded offline execution for irrigation |
| Status | Proposed |
| Context | Reliability/Safety and Availability are driving QAs. ASR5 forbids anomalous readings triggering flooding; ASR12 requires controllers to keep the last approved schedule for 48 hours offline; C3 requires 72-hour edge operation. Faulty sensors, invalid requests or outdated schedules could waste water or damage crops. Alternatives: cloud-only control stops watering during rural outages; unrestricted local autonomy continues unsafe or outdated schedules indefinitely; executing requests immediately removes human oversight. A disconnected controller cannot receive cloud cancellations, so offline safety must be enforced locally. |
| Decision | Irrigation requests start PENDING and run only after owner approval. Validate units and configurable safe bounds at creation, approval and controller execution; defaults are 1-240 minutes and 0.5-50 mm per request. Only accepted, non-quarantined readings may inform schedules. Approved schedules carry an expiry and a 48-hour offline lease; controllers run them locally during outages and honour local overrides. When the lease lapses without cloud contact, controllers stop automatic watering and revert to manual control. The prototype implements PENDING creation, pending-only replacement and safe-limit validation. |
| Consequences | Watering continues through common outages, while invalid requests and faulty readings cannot trigger unbounded irrigation. Approval delays watering. Cloud cancellations cannot reach disconnected controllers until reconnection; expiry and local overrides bound that risk. Safe-state shutdown may under-water during long outages. Per-paddock limits need configuration and agronomic validation. |
| Compliance | Test out-of-limit and non-pending requests return 422 and 409. Simulate 48-hour disconnection; confirm local execution, expiry shutdown and manual override. Audit that unapproved schedules never reach controllers. |

**Traceability:** ASR5, ASR10, ASR12, ASR14, ASR21; constraint C3; Reliability / Safety and Availability driving QAs. Telemetry quarantine rules follow Aditya's ADR-02 and the team integration contract.

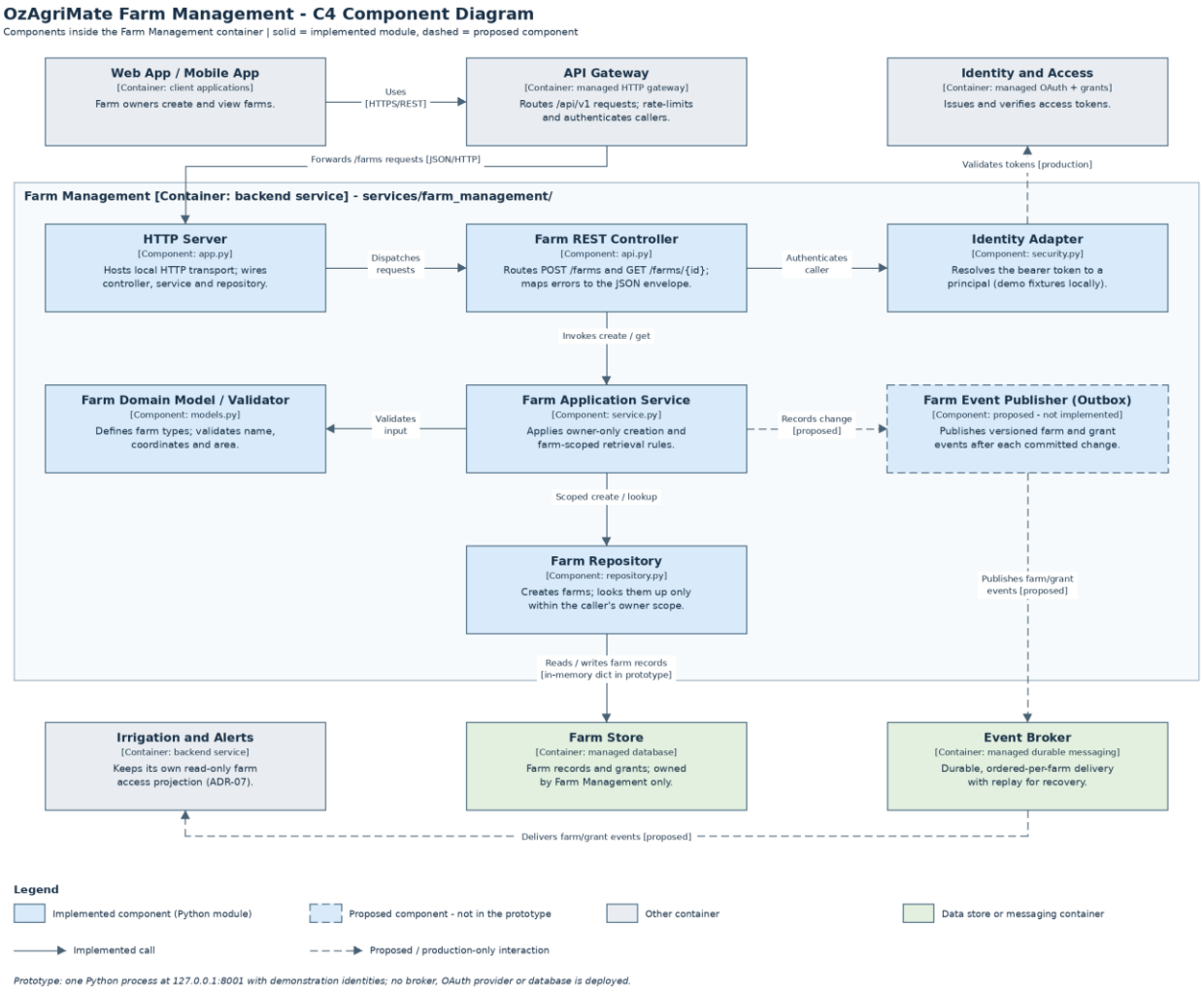
## ADR 07 Farm access projection

| Field | Architectural decision record |
|---|---|
| Title | ADR-07 Maintain an event-fed farm access projection inside Irrigation and Alerts |
| Status | Proposed |
| Context | Irrigation and Alerts must confirm farm existence and caller access on every request, but Farm Management owns farms and grants in its own store (ADR-01). ASR19 requires revocation within one minute; ASR26 forbids cross-farm leakage. Alternatives: a shared farm database couples schemas and releases and breaks data ownership; a synchronous call per request adds latency and makes irrigation unavailable whenever Farm Management fails; long-lived response caches risk acting on revoked access. A local copy is safe only if its staleness is bounded and detectable. |
| Decision | Farm Management records farm and grant changes in an outbox and publishes versioned, per-farm-ordered events through the Event Broker. Irrigation and Alerts applies them to its own read-only projection: duplicate event IDs are ignored, older versions are discarded, and version gaps trigger resynchronisation from a snapshot. Source heartbeats advance a freshness watermark. New protected mutations return 503 when the watermark is older than 60 seconds or resynchronisation is pending. The prototype seeds the projection from fixture events and implements the deduplication, ordering, gap and freshness rules. |
| Consequences | Irrigation keeps working through brief Farm Management outages without shared tables or per-request coupling. Eventual consistency means mutations may be refused during broker delays, reducing availability. The outbox, heartbeats, snapshots and replay add operational complexity. One-minute revocation depends on measured broker latency and remains unvalidated in production. |
| Compliance | Automated tests cover revoked grants, duplicate, out-of-order and gap events, and stale watermarks. Production monitoring alerts when projection lag exceeds 30 seconds. |

**Traceability:** ASR19, ASR26; ADR-01 service-owned stores; ADR-05 farm-scoped authorisation. Requires the Farm Management outbox shown as proposed in Figure 3.

# Farm Management component view

**Figure 3 OzAgriMate Farm Management C4 Component Diagram**



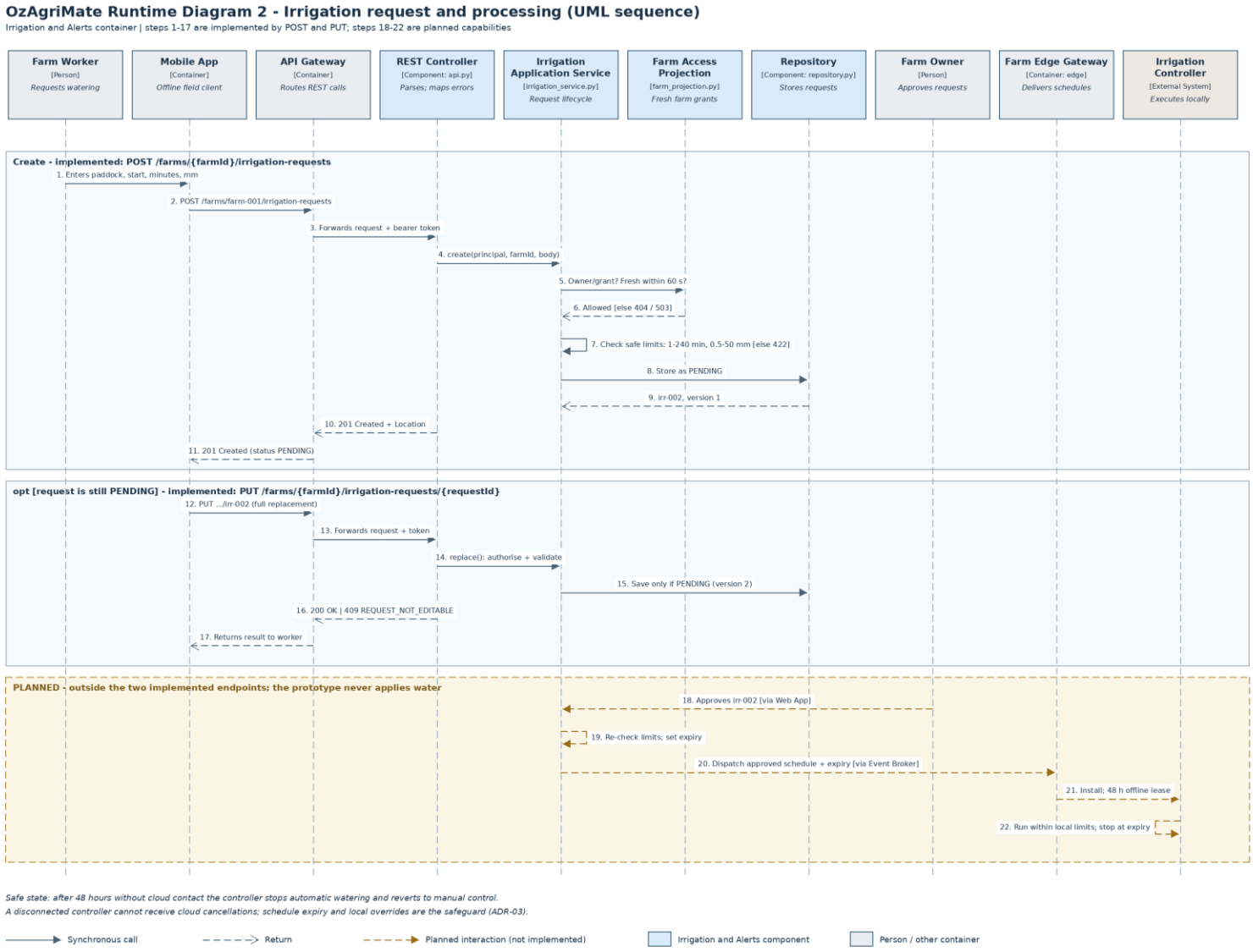

The view decomposes the Farm Management container into the modules Pulkit implemented, so the repository structure and diagram correspond one-to-one. The Farm Event Publisher is drawn dashed because the farm prototype does not yet publish events; it is required by ADR-07.

| Component | Module | Responsibility |
|---|---|---|
| HTTP Server | app.py | Local HTTP transport and composition root |
| Farm REST Controller | api.py | Routes the two farm endpoints; maps errors to the JSON envelope |
| Identity Adapter | security.py | Resolves the bearer identity (demonstration fixtures locally) |
| Farm Application Service | service.py | Owner-only creation and farm-scoped retrieval |
| Farm Domain Model / Validator | models.py | Farm types and request validation |
| Farm Repository | repository.py | In-memory storage with owner-scoped lookup |
| Farm Event Publisher (Outbox) | Proposed | Publishes versioned farm and grant events after commit |

# Runtime view: irrigation request and processing

**Figure 4 OzAgriMate Runtime Diagram 2: irrigation request and processing (UML sequence)**

**Interaction flow:** Worker creates and edits a validated PENDING request; planned owner approval sends a time-limited schedule to the field controller.



# Irrigation API design

Paths follow the team integration contract. Production versioning is proposed as /api/v1 at the gateway; the prototype exposes the unprefixed service paths at http://127.0.0.1:8003.

## POST /farms/{farmId}/irrigation-requests

| Specification | Value |
|---|---|
| **HTTP Method** | POST |
| **Description** | Create a pending irrigation request for a farm paddock |
| **Endpoint URL** | /farms/{farmId}/irrigation-requests |
| **Sample Request Body** | {<br>"paddockId": "north-paddock",<br>"scheduledStart": "2026-11-02T19:00:00Z",<br>"durationMinutes": 90,<br>"waterDepthMm": 18.5,<br>"notes": "Pre-heatwave watering"<br>} |
| **Sample Response Body** | {<br>"id": "irr-002",<br>"farmId": "farm-001",<br>"paddockId": "north-paddock",<br>"scheduledStart": "2026-11-02T19:00:00Z",<br>"durationMinutes": 90,<br>"waterDepthMm": 18.5,<br>"notes": "Pre-heatwave watering",<br>"status": "PENDING",<br>"requestedBy": "worker-a",<br>"version": 1<br>} |
| **Success status** | 201 Created, with a Location header |

## PUT /farms/{farmId}/irrigation-requests/{requestId}

| Specification | Value |
|---|---|
| **HTTP Method** | PUT |
| **Description** | Replace a pending irrigation request's editable details |
| **Endpoint URL** | /farms/{farmId}/irrigation-requests/{requestId} |
| **Sample Request Body** | {<br>"paddockId": "south-paddock",<br>"scheduledStart": "2026-11-03T18:30:00Z",<br>"durationMinutes": 60,<br>"waterDepthMm": 12<br>} |
| **Sample Response Body** | {<br>"id": "irr-002",<br>"farmId": "farm-001",<br>"paddockId": "south-paddock",<br>"scheduledStart": "2026-11-03T18:30:00Z",<br>"durationMinutes": 60,<br>"waterDepthMm": 12,<br>"notes": "",<br>"status": "PENDING",<br>"requestedBy": "worker-a",<br>"version": 2<br>} |
| **Success status** | 200 OK |

The irr-002 example assumes the first creation after server startup; irr-001 is a seeded APPROVED request used to demonstrate the lifecycle conflict. Consumers should use the returned ID. PUT is a full replacement: an omitted notes field becomes empty, and the version increases by one.

### Validation and error behaviour

A body must contain paddockId, scheduledStart, durationMinutes and waterDepthMm; notes is optional. Any other field, including caller-supplied id, status, requestedBy or owner fields, is rejected. paddockId is 1 to 60 lowercase letters, digits or hyphens. scheduledStart is a valid UTC timestamp such as 2026-11-02T19:00:00Z. The ADR-03 safe limits apply: durationMinutes is a whole number from 1 to 240 and waterDepthMm is a number from 0.5 to 50. notes is at most 500 characters. Booleans, nulls and numeric strings are rejected.

Demonstration identities match Farm Management. demo-owner-a owns farm-001 and demo-worker-a holds an operations grant on it, so both may create and replace requests there. demo-owner-b owns farm-002 and cannot see farm-001. farm-003 belongs to owner A but its event source is deliberately stalled to demonstrate the ADR-07 freshness rule.

| Status | Code | Relevant condition |
|---|---|---|
| 400 | INVALID_JSON | Malformed UTF-8 JSON or literal NaN/Infinity |
| 401 | UNAUTHENTICATED | Missing or invalid demonstration bearer identity |
| 404 | FARM_NOT_FOUND | Farm absent or not accessible to the caller (identical bodies) |
| 404 | IRRIGATION_REQUEST_NOT_FOUND | Request absent or belongs to another farm |
| 404 | ROUTE_NOT_FOUND | Unknown path or malformed identifier |
| 405 | METHOD_NOT_ALLOWED | Unsupported method; Allow header names the supported one |
| 409 | REQUEST_NOT_EDITABLE | PUT on a request that is no longer PENDING |
| 413 | BODY_TOO_LARGE | Body exceeds 65,536 bytes |
| 415 | UNSUPPORTED_MEDIA_TYPE | Content-Type is not application/json |
| 422 | VALIDATION_ERROR | Missing/extra fields, invalid values or safe limits exceeded |
| 503 | FARM_ACCESS_STALE | Farm access cannot be confirmed within 60 seconds; Retry-After: 5 |

**Error response example:**

```
{"error":{"code":"REQUEST_NOT_EDITABLE","message":"Only PENDING requests can be replaced; this request is APPROVED."}}
```

### Where the endpoints sit in the architecture

Both endpoints are served by the Irrigation and Alerts container. In production, the Web or Mobile App calls the API Gateway, which routes them to this service. The REST Controller delegates to the Irrigation Application Service, which checks the Farm Access Projection rather than Farm Management's store, validates ADR-03 limits and persists through the Repository to the service's Operations Store partition. No AI Recommendation dependency exists, so irrigation requests continue during AI outages (ASR21).

## Repository and component alignment

The code lives in services/irrigation_alerts/, matching the module names agreed with Bradley for the Irrigation and Alerts component view.

| Component | Module | Status |
|---|---|---|
| REST Controller (shared routing) | api.py | Implemented; irrigation routes |
| Irrigation Application Service | irrigation_service.py | Implemented (Thomas) |
| Alert Rule Application Service | alert_rule_service.py | Placeholder (Bradley) |
| Farm Access Projection | farm_projection.py | Implemented; ADR-07 rules |
| Repository | repository.py | Implemented; in-memory |

| Component | Module | Status |
|---|---|---|
| Domain Model / Validator | models.py | Implemented; ADR-03 limits |
| Identity Adapter | security.py | Implemented; demonstration identities |
| Farm Event Consumer | farm_events.py | Proposed; local fixture source only |
| HTTP Server | app.py | Composition root, port 8003 |

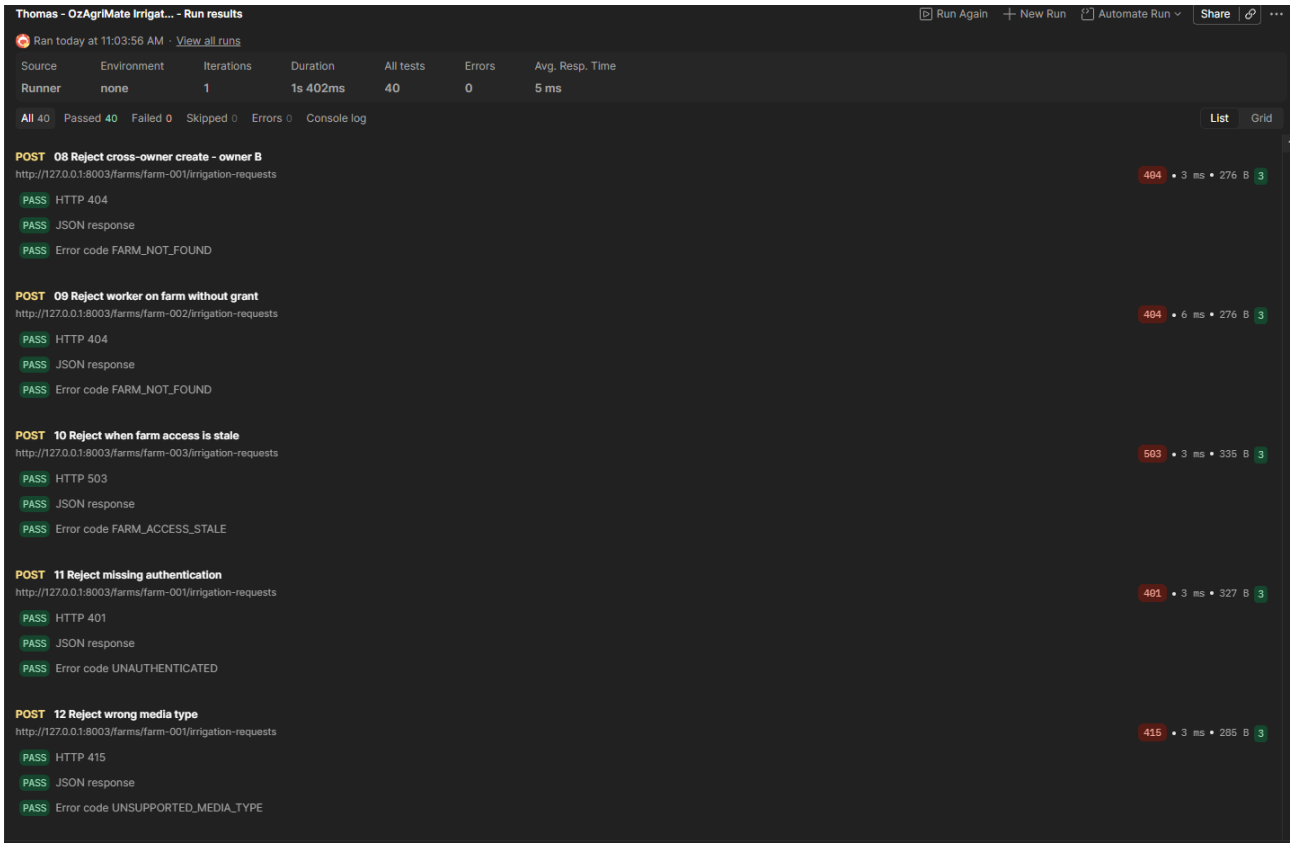
## Validation evidence

Twenty-one automated tests run against the code. Fifteen HTTP integration tests start a real local server and cover the exact create and replace contracts, owner and worker access, cross-owner denial, the stale-projection 503, rejected caller-supplied fields, safe-limit boundaries, partial PUT bodies, the 409 lifecycle conflict, missing requests, authentication, media type and method handling. Six unit tests cover ADR-07 projection rules: revocation, duplicate events, out-of-order replay, version gaps and watermark expiry. The full repository suite, including Pulkit's ten farm tests, passed 31 of 31.

| Measure | Observed result |
|---|---|
| Automated tests (irrigation) | 21 passed, 0 failed |
| Full repository suite | 31 passed, 0 failed |
| Postman collection | postman/Thomas_Irrigation.postman_collection.json, 13 requests, 40 assertions |
| Postman run (Collection Runner, 1 iteration) | 40 assertions passed; 0 failed, 0 skipped, 0 errors |
| Displayed duration / average response | 1 second 402 milliseconds / 5 milliseconds |

Thomas ran the collection in the Postman desktop Collection Runner against the local service at 127.0.0.1:8003 on 24 October 2026 at 11:03:56 AM. Figure 5 shows the run summary and representative error scenarios. The run confirms creation, pending-only replacement, the 409 lifecycle conflict, cross-owner and ungranted denial, the stale-projection 503, safe-limit validation, authentication, media type and method handling.

**Figure 5 Postman Collection Runner result for the irrigation endpoints**



The original screenshot is retained as evidence/Thomas_Postman_Run_2026-10-24.png. The reported latency belongs to one local functional run and is not evidence of performance at 50,000 farms. Local results also do not demonstrate production durability, broker latency, one-minute revocation or offline controller behaviour.

## References


[1] Brown, S. (n.d.). Component diagram. C4 model. https://c4model.com/diagrams/component. Accessed 2 1 October 2026.

[2] Brown, S. (n.d.). Dynamic diagram. C4 model. https://c4model.com/diagrams/dynamic. Accessed 2 1 October 2026.

[3] Sharma, P., Singh, A., Wight, T., and Shears, B. (2026). OzAgriMate Software Architecture Proposal. Submitted ISYS3459 Assignment 2 report.

[4] Sharma, P. (2026). OzAgriMate Architecture and Farm API; Team Integration Contract. ozagrimate-a4 repository.

[5] RMIT University. (2026). ISYS3459 Assignment 4 Software Architecture Project activity instructions. Supplied course material.

[6] RMIT University. (2026). ADR Template. Supplied course material.